Departamento de Computación
Facultad de Ciencias Exactas y Naturales
Universidad de Buenos Aires

INFORME TÉCNICO

OLAP, Data Warehousing, and Materialized Views : a Survey

Alejandro Ariel Vaisman

Report n.: 98-015

Pabellón 1 - Planta Baja - Ciudad Universitaria
(1428) Buenos Aires
Argentina

http://www.dc.uba.ar

Title: OLAP, DATA WAREHOUSING AND MATERIALIZED VIEWS: A SURVEY

Authors: Alejandro Ariel Vaisman

E-mail: av2n@dc.uba.ar

Report n.: 98-015

Key-words: OLAP, databases, views, SQL, data warehousing

Abstract: This report aims to give a comprehensive introduction to the subjects of Data Warehousing and OLAP. It also gives an overview of the related topic of Materialized Views. Materialized Views become important when trying to improve the performance of an OLAP system. The main intention is to describe the state of the art in these fields in order to identify research opportunities. Topics covered here are: architecture, design, querying, optimization and implementation. Relevant research papers are commented, and some commercial products reviewed, mainly to remark their differences. ROLAP and MOLAP are also analyzed and, finally, an extensive bibliography is presented in the references.

To obtain a copy of this report please fill in your name and address and return this page to:

Infoteca
Departamento de Computación - FCEN
Pabellón 1 - Planta Baja - Ciudad Universitaria
(1428) Buenos Aires - Argentina

TEL/FAX: (54)(1)783-0729
e-mail: infoteca@dc.uba.ar

You can also get a copy by anonymous ftp to: **zorzal.dc.uba.ar/pub/tr**
or visiting our web: **http://www.dc.uba.ar/people/proyinv/tr.html**

Name:.....

Address:.....

.....

OLAP, DATA WAREHOUSING AND MATERIALIZED VIEWS: A SURVEY

Alejandro A. Vaisman
University of Buenos Aires
e-mail : av2n@dc.uba.ar

ABSTRACT

This report aims to give a comprehensive introduction to the subjects of Data Warehousing and OLAP. It also gives an overview of the related topic of Materialized Views. Materialized Views become important when trying to improve the performance of an OLAP system. The main intention is to describe the state of the art in these fields in order to identify research opportunities. The topics covered here are: architecture, design, querying, optimization and implementation. Relevant research papers are commented, and some commercial products reviewed, mainly to remark their differences. ROLAP and MOLAP are also analyzed and, finally, an extensive bibliography is presented in the references.

1. INTRODUCTION

Since the late seventies, Relational Database technology has been gaining widely acceptance, to the length that today, most of the organizations rely on it to storage their data. However, the needs of these organizations are not the same as they used to be. Increasing markets´ dynamics and competitiveness are leading to the necessity to have the right information at the right time. Managers need to be properly informed, in order to take appropriate decisions for running business. On the other hand, data possessed by such organizations are usually scattered among different systems, each one devised for a particular kind of job. Moreover, these data are stored in different systems. In this way, data otherwise valuable, become lost. We can say that we have data, not information.

Traditional systems, based on transaction processing, are not well - suited for the above mentioned new requirements. This gave rise to new database technologies, called **Data Warehousing** and **OLAP**, and, in general, to what are called **Decision Support Systems**(DSS).

In this section we will explain why transactional systems are not appropriated for decision support, and which are the latter´s special characteristics. We will also give some definitions for the most commonly used terms.

Data Warehousing refers to architectures, algorithms, tools and techniques for bringing together data from multiple databases or other information sources, into a single repository suited for querying or analysis. This repository is called a **Data Warehouse**. As we outline above, it is important for organizations to gather all their data into a single place, so analytical processing can be done without

interfering with their operational systems. These systems are usually referred as **On Line Operational Processing (OLTP).** The goal for these systems is to get the maximum number of transactions per second. Decision support systems perform more complex queries. They are concerned with historical data, and usually involve the computation of aggregation and statistical functions. Typical queries are of the form : "What was the total sales of toys in Christmas for each of the last five years in California?", maybe followed by "give me the four best-selling toys in the last five Christmas in California". Systems which are able to perform queries like these, within a reasonable response time, enabling high-level users like managers and business analysts to navigate easily through enterprise data, giving a multidimensional view of these data, belong to what is called **On Line Analytical Processing (OLAP)**.

We summarize the differences between OLTP and OLAP,in the following table:

	OLTP	**OLAP**
Typical user	regular employee	manager, analyst
Usage of system	day to day operation	business analysis
User interaction	pre-determined	ad-hoc
Data	current data	current data
Data characteristics	atomic	summarized
Work characteristics	read/write	read (exept off-line updates)
Unit of work	transaction	query
Processing	process-oriented	subject-oriented
Updates	One record at a time	Several records at a time

We can say that the main characteristics of OLAP systems are :

- they give a multidimensional view of the data, no matter how it is actually stored.

- always involve a graphical-interactive query.

- allow drill-down, roll-up, and "slice and dice" of data, so users can analyze information along different dimensions and in various depth levels.

- Quick response time (no more than two minutes, in general).

In the following sections, we describe how these goals are achieved. In **section 2**, we review the characteristics of OLAP systems and data warehouses, pointing out the differences between ROLAP and MOLAP (Relational and Multidimensional OLAP, respectively). **Section 3** presents a short description of materialized views, which are an important way of improving performance. We need this in order to understand how data warehouses are implemented. In **section 4** we give an in-depth treatment to the topic of data warehouse implementation, whereas in **section 5** we deal with indexes and physical implementation. We conclude in **section 6** and present a final reference section.

2.- DATA WAREHOUSING AND OLAP

2.1.-DATA WAREHOUSING

Roughly speaking, a **Data Warehouse** is a database that gathers data coming from different sources of an organization. By **Data Warehousing** we mean a series of processes which turn raw data into data suitable to be queried, in order to facilitate the process of decision making.

Data Warehouses should be separated from the operational data, for a series of reasons. Here are some of them :

- Operational systems are designed and tuned in order to answer efficiently answering queries like “Find the balance of the accounts belonging to John Smith”. Entity-Relationship modeling is widely used in designing operational databases. The resulting database is usually highly normalized. However, the Entity-Relationship model is not suited for Data Warehouse design, so new models were developed, the **Star-Schema** being the most popular. In order to handle OLAP queries, the structure of the Data Warehouse must be as simple as possible.

- Operational systems are usually working at the limit of their capacity, so loading them with heavy queries would be unacceptable.

- We will also see that indexing and query processing techniques are different.

- Operational systems are constantly being updated, while Data Warehouses are read-only databases, except during off-line updates, so transaction management is completely different.

The **main processes** within a Data Warehousing environment are :

1. Data Extraction and Loading.
2. Data Cleaning and Translating.
3. Backup and Archiving.
4. Query Management.

1. **Data Extraction.** It is the process of obtaining data from its sources and making it available for the Data Warehouse. The **Loading** process places data into the Data Warehouse. Before loading can take place, data coming from different operational sources should be made consistent. Data extracted from the sources, is usually loaded into temporary files. All further cleaning processes are performed on these files.
It is important to realize that data usually come from heterogeneous sources like Relational Databases, ASCII files, Legacy Systems, Purchased data, etc. These data must be integrated, filtered and translated into the format of the Data Warehouse. So the system must be able to manage different data models.

2. **Data Cleaning and translating.** The steps involved in this process are:
- Clean and translate loaded data into a data structure appropriate for querying.
- Partition data in order to make data management easier and improve performance.

- Create aggregations for the most common queries.

Cleaning goals are making data

- self-consistent(check for incorrect data).
- consistent with other data from the same source.
- consistent with data from other sources (i.e. check a customer record to see if it is consistent with another record at a different source representing the same customer).

The next step will be to bring cleaned data into a structure suited for analytical processing (the Star Schema).

3.**Backup and Archiving.** This is similar to the OLTP process.
4.**Query Management.** This is the query processing stage. We will talk about it later.

A typical **Process Architecture** is depicted in the following schema.

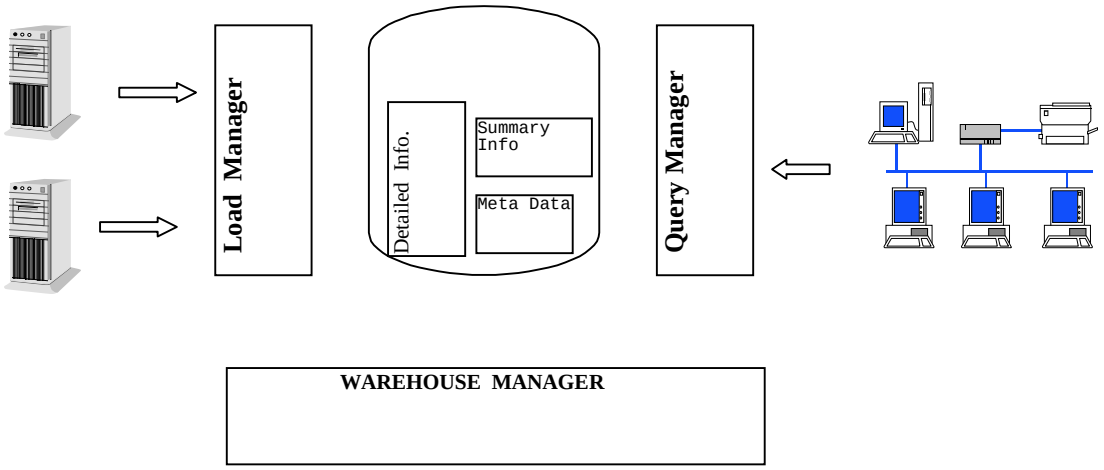


Figure 1

The functions each module performs are:

Extraction and Loading ------> Load Manager
Cleaning, Translating,
Backup and Archiving ------> Warehouse Manager
Query Management ------> Query Manager

Load Manager

This is the module that involves data extraction and fast loading into temporary files.
It can be an Off-the-shelf product, or a collection of C/C++ programs and Stored Procedures.

Warehouse Manager

Its tasks are :

- Analyze data, checking integrity and consistency.
- Translate and merge data in the temporary structure into the Data Warehouse structure.
- Generate the Star/Snowflake Schema.
- Create Views, Indexes, etc.
- Create and update Aggregations.
- Backup and Archive data.
- Analyze the query profile.

Query Manager

This is the module which supports query processing. It must evaluate queries efficiently using views, indexes and aggregations available.

Query Manager

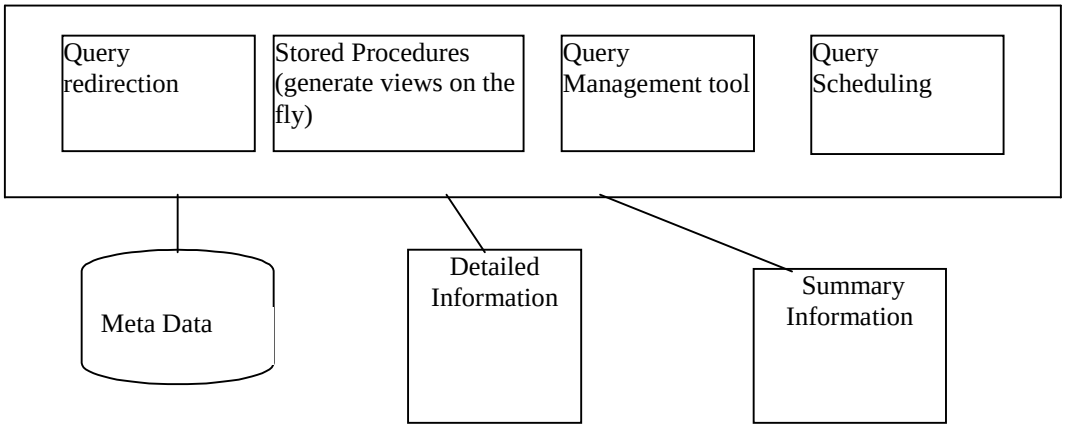


Figure 2.

Before getting into the issue of how to design a Data Warehouse, let us describe what kind of information is stored in it. We are going to refer to figure 2.

Detailed Information

This is where the detailed information for the Star Schema(see below) is stored. It is not necessarily On-Line information. Usually, On- Line information is summarized to the next level of detail, and detailed information is stored on a tape device, and loaded as needed. For instance the detailed table :

Tid	Prod_id	Store_id	day	sales_amt
1	p1	s1	1997-9-20	200
2	p1	s1	1997-9-20	200
3	p2	s1	1997-9-20	300

would be stored Off-Line, while the following will be the On-Line table :

Prod_id	Store_id	day	total_sales
p1	s1	1997-9-20	400
p2	s1	1997-9-20	300

The decision will be based on the answer to the following question : "Does any business process require the analysis of individual transactions?". If the answer is "yes", maybe we do just need to store the last three months of individual transactions.

Summarized Information

This is the area of the Data Warehouse where predefined aggregations are stored. We will talk about summary tables in the following sections. At this time, we just wish to point out that summarized information

- enhances query performance
- increases operational costs
- must be updated each time new data are loaded
- could not be backuped, if desired

As an empirical value, the number of summary tables should not exceed two hundred. During design, space for these tables must be reserved.

Metadata

This is the area where data definition is stored. Every process uses this data. During Loading and Extraction, to map sources to the data structure of the Warehouse. The Warehouse Manager uses Metadata in the creation of Summary Tables. Finally, the Metadata is used by the Query Manager to access the adequate tables.

Data Warehouse Design

As queries submitted to Data Warehouses are of a different nature than the ones sent to an operational database, the data model supporting the repository should be also be different. The Entity - Relationship Model, although adequate for operational databases, leads to a highly normalized database, which is not recommended for a Data Warehousing environment. Thus, a simpler model is commonly used, which is known as the **Star Schema.** This model presents data in a fashion which resembles the way users view it in their business. Data is organized in two kinds of tables : the **Fact Table** and the **Dimension Tables.** Obviously, facts do not change across time, while dimensions do. This characteristic is exploited by the model. Users analyze data through dimensions, like Customers, Products, etc. These dimensions can change across time. On the other hand, facts like units sold on a particular day, do not change.

Fact tables usually occupy hundreds of Gigabytes, so they must be carefully designed. Besides, they are the only normalized tables in the Data Warehouse. Dimension tables are denormalized and smaller, although some of them (called Big Dimensions) would have several millions of records.

Fact tables

The main goal in Fact table design is to get the smallest possible table without losing information.
Some guidelines are :
- Identify the interesting elementary transactions of the business in question.
- Identify the data retention period. time window for analysis.
- Define the table granularity. It is possible that we do not need to store every transaction. In this case, some aggregation must be performed. For instance, we may want to store the daily sales of each product in the retail store, not every sale made.
- Minimize column size.
- Dimension tables join to the Fact table through Foreign Keys. Use FK which do not change over time.
- Care must be taken with some facts which can not be added across every dimension. An example are account balances. These are called semi-additive facts.
- Normalize the Fact tables.
- Fact tables should hold about 70% of the total size of the Data Warehouse.
- Fact tables can partitioned in order to improve performance. This partitioning is made according to time periods. Fact data for each period is stored in a different table, but keeping in mind that the overall number of tables in the Data Warehouse should not exceed five hundred. Of course this is just empirical.
Partitioning improves not only query performance, but also makes administrative tasks (like backup) easier. Queries comparing time periods (i.e. sales this quarter against same quarter last year), benefit from this technique. Its importance can be summarized by a popular saying which states : Partition or die!.

Examples of Fact tables are :

Retail business	**TV-transactions**	**Bank accounts**
Product_id	Customer_id	Customer_id
Store_id	Channel_id	Account_number
Time_id	Time_id	Transaction_type
qty	Program_id	Amount
benefit	Duration	

Dimension Tables

Dimension tables describe business characteristics. Denormalization is the main technique in their design. They are organized in hierarchies, which most systems handle, in order to improve roll-up and drill-down operations.

One problem arises when dimensions can change over time. There are two main solutions for this case : overwrite the old values (the option in most commercial systems) or maintaining versions, creating a new tuple each time a dimension attribute changes.

When dealing with Big Dimensions, some authors [STG] recommend to normalize these tables, which originates the **Snowflake Schema**. This solution, however, is strongly discouraged by R.Kimball[K96].

Some examples of Dimension Tables are :

Product	**Store**
Prod_id	Store_id
Section	name
Department	number
Business_Unit	City
Product_name	Country
Color	Region

Hierarchies are:

Department ---> Section -----> Business_Unit
Region ----> Country -----> City

An example of a Star Schema is shown in the following figure.

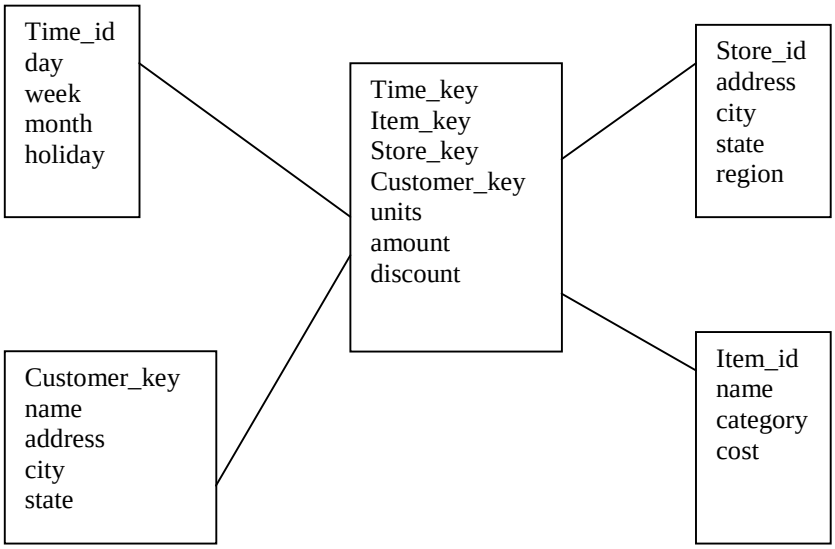


Figure 3.

2.2.-OLAP

As the OLAP Council White Paper[OCWP] states : "OLAP enables analysts, managers an executives to gain insight into data, through fast, consistent, interactive access to a wide variety of possible views of information". OLAP transforms data stored in a Data Warehouse into valuable information. In this way, OLAP and Data Warehouses are complementary. Data Management is performed by the Data Warehouse and Data Analysis is carried out by OLAP.

In [CCS93] twelve rules which should be accomplished by an OLAP system are introduced, considering characteristics such as data access, data definition, user requirements and front-end accessibility. These rules are :

1. **Multi-Dimensional Conceptual View**. Allows the users to view data in the way he view his business.

2. **Transparency.** The place where the OLAP system resides should be transparent to the user.
3. **Accessibility.** The system from where data is obtained should be transparent to the user. The OLAP tool should handle possible heterogeneous data.
4. **Consistent Reporting Performance.** As the number of dimensions increases, performance should not degrade.
5. **Client-Server Architecture.**
6. **Generic Dimensionality.** Every dimension should be equivalent, in structure and operational capabilities.
7. **Dynamic-Sparse Matrix Handling.** The system should adapt to any data configuration, dynamically changing, if needed, its data access methods.
8. **Multi-User Support.**
9. **Unrestricted Cross-Dimensional Operations.** Any cell may be used in any formula.
10. **Intuitive Data Manipulation**. Drill-down, Roll-up, etc., should be performed in an intuitive way, never through a menu or similar.
11. **Flexible Reporting.**
12. **Unlimited Dimensions and Aggregation Levels.**

OLAP systems provide a multidimensional view of the data, no matter how this data is physically stored. Data is perceived by the user as a **multidimensional cube** where each cell contains a value or **measure**. According to the actual physical architecture, there are two mainstreams: **ROLAP** and **MOLAP**, standing for **Multidimensional** and **Relational** OLAP respectively. In ROLAP, data is stored in Relational Databases, in tables following the Star Schema, and an intermediate module translate them into a "cube", which is what the user sees. In MOLAP, data is stored in proprietary arrays, specifically conceived for dimensional analysis. As vector arithmetic is applied in computing aggregation, MOLAP delivers better performance, although it has some drawbacks. Typical architectures are depicted in figure 4.

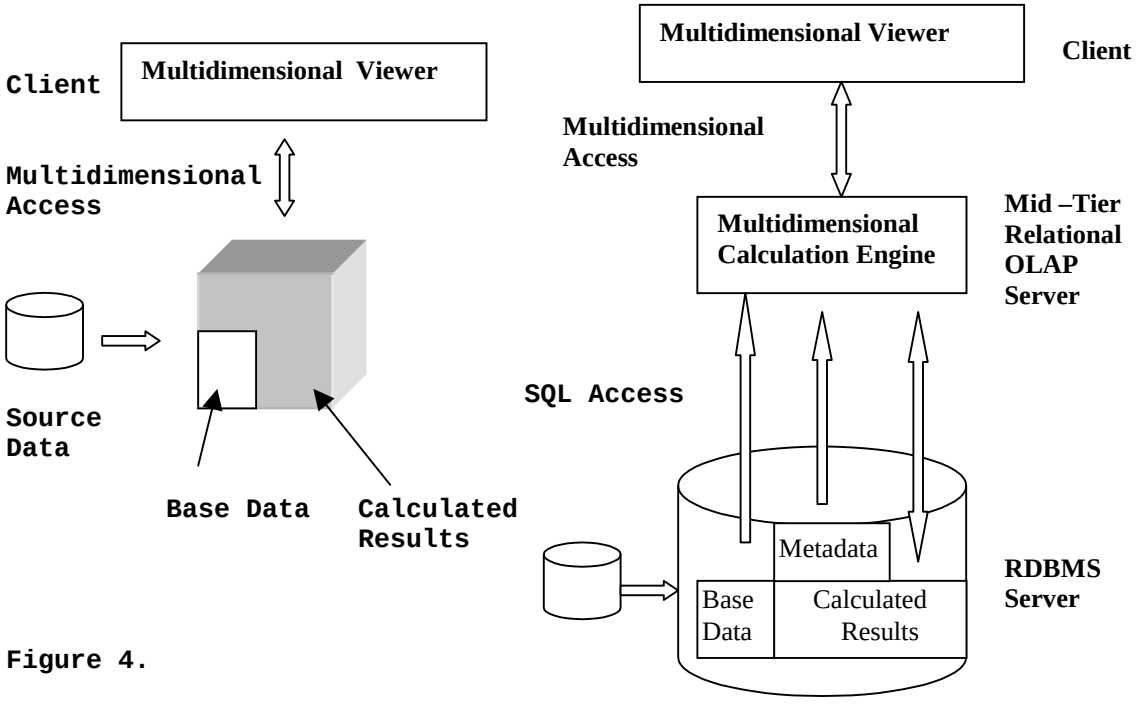


Figure 4.

MOLAP servers apply compressing strategies in for handling sparse cubes(cubes in which most of the cells have no value), make an

efficient use of caching, and precompute aggregates. On the other hand, this consumes a lot of space. As a consequence, MOLAP is mostly used in Data Marts (small Data Warehouses involving just a single sector of an organization).

The following table points out some important differences between MOLAP and ROLAP architectures.

	ROLAP	MOLAP
Storage and Access	-Tables/tuples -SQL access language -Third party tools	-Propietary arrays -Lack of a standard language -Sparse data compression
Usage	-Variable performance -Relational engine	-Good performance -Multidimensional engine
Database size	-Gb to Tb -Large space for indexes -Easy updating	-Gb -2% index space -Difficult updating

We will explain further the differences between OLAP AND ROLAP.

- **A. Physical Storage.**

  MOLAP.-

  - Summarization is performed by the multidimensional engine.
  - Data is stored in arrays.
  - Only cells with actual values are stored.
  - Indexes are just necessary for sparse data management, and fit in main memory.

  ROLAP.-

  - Summarization is performed at an external engine.
  - Data is mapped into tuples of the Star Schema tables.
  - Aggregates are precomputed in summary tables.
  - Metadata is stored in relational tables.

- **B. Data Management.**

This is the layer which controls data access, and manages physical structure and security.

  MOLAP

  - Locking at the cell level.
  - This layer manages dimension hierarchies.

  ROLAP.-

  - All data management relies on the underlying RDBMS.

- **C. Multidimensional Manipulation.**

MOLAP.-

- Rotation and Slice and Dice flexibility.
- Great computing capacity across every dimension.

ROLAP.-

- Limitations imposed by SQL.
- Most computation is performed outside the engine.
- Rankings, moving averages, and other functions, are not supported by SQL. These functions are computed either at the Client or at the OLAP engine.

- **D. Performance.**

In general, ROLAP brings a larger storing capacity, but this increases space required for indexes and maintenance costs. On the other hand, as MOLAP precomputes everything, it delivers a better performance. ROLAP systems must store summary tables as the clue for bringing acceptable response time.

Loading in ROLAP is faster than in MOLAP because data comes from sources with a similar structure, while MOLAP systems must first sort data in order to enable an efficient loading.

E. State of the practice.

Most Relational Database vendors offer ROLAP products. Other vendors offer multidimensional engines. In the references, we cite these vendors´ White Papers.

3.-MATERIALIZED VIEWS

Before getting into the details of how Data Warehouses can be implemented, it becomes necessary to describe what Materialized Views are , and how they relate to Data Warehousing.

We can perceive a Data Warehouse as a view of data stored in the sources[LW95]. This views are not the conventional ones, but views which are actually materialized in the repository, usually called, as we have already seen, Summary Tables. Thus, maintaining a Data Warehouse becomes a special case of the problem of view maintenance.

We know a **View** is a derived relation defined in terms of base relations. A view defined in this way(typically created by means of the CREATE VIEW statement), is recomputed every time it is invoked .

A **Materialized View** is a view such that its tuples are stored in the database. This improves query performance, playing the role of a cache, which can be directly accessed without looking into the base relations. But this benefit has a counterpart. When the base data is updated, the materialized views derived from that data also needs to be updated. The process of updating a materialized view in response to changes in the base data, is called **view maintenance.** This process not always has to involve recomputing the entire view from scratch. Sometimes it is possible just compute changes to the view caused by changes in the underlying relations. This is called **incremental view maintenance**. As this problem is central to Data Warehousing, we will focus on it.

The View Maintenance problem has four dimensions through which it may be analyzed :

- **Information**: It refers to the available information for view maintenance, like integrity constraints, keys, access to base relations , etc..
- **Modification:** Meaning what modifications can be handled by the maintenance algorithm. The algorithms should handle deletions, insertions an updates, usually treated as a deletion followed by an insertion.
- **Language:** About which language is used to define the query, most often SQL. Aggregation is another issue, as well as recursion.
- **Instance:** Whether or not the algorithm works for every instance of the database, or for a subset of all instances.

As an example, let us suppose a relation

Assigned_to(emp_no, proj_no, hourly_salary)

and the view

$$\text{Well\_paid(emp\_no)} = \pi_{\text{emp\_no}}\, \sigma_{\text{hourly\_salary}>150}(\text{Assigned\_to})$$

It is clear that **inserting** a tuple like (e20,pA14,110) would make no effect on the view, but insertion of (e20,pA20,160) would modify it. However, if the materialized view is available, an algorithm can update it without accessing the base relation .

In case of a **deletion**, say of tuple (e20,pA20,160), the situation is analogous. We cannot delete e20 from the view until checking if e20 is not assigned to another project with an hourly salary greater than U$S 150, which can not be done unless we access relation Assigned_to.

In summary, although in some cases insertion can be performed just accessing the materialized view, deletion always requires further information.

Consider now a view involving a join.

$$\text{CAL\_EMP} = \pi_{\text{emp\_no}}\,(\sigma_{\text{state= ``CA''}}(\text{Projects}) \,|x|\, \text{Assigned\_to})$$

If we insert the tuple (e23,pC30,170), we cannot know if e23 will be in the view unless we check in the base relations. The exception would be if e23 already belongs to the view. This is always the case when we have a join condition.

We will review two situations :

- Algorithms using Full Information: base relations and views. This case applies to every kind of query, and every instance.
- Algorithms using partial Information. This information will be the Materialized Views and the Key Constraints.

A.- Concerning the case of using **Full Information**, we will review three kinds of views: Non-recursive Views, Outer-Join Views, and Recursive Views.

a1. The Counting Algorithm. Applies to **non-recursive** SQL views, which may be defined using UNION, negation and aggregation. The basic idea is to count the number of alternative derivations every tuple in the view

has. In this way, if we delete a tuple in a base relation, we can check if we should delete it from the view or not.

Consider the view

```
CREATE VIEW One_Stop as
   SELECT DISTINCT city1.From, city2.To
      FROM Connected City1, Connected City2
      WHERE City1.To = City2.From
```

and this instance for relation **Connected**:

From	To
a	b
b	c
b	e
a	d
d	c

The view One_Stop will be(included the number of derivations for each tuple, named **Count**)

From	To	Count
a	c	2
a	e	1

Suppose we delete tuple (a,b). Although (a,c) is derived from (a,b), it has 2 alternative derivations.

If we apply differenciation to the expression defining the view, we get(Conn stands for Connected) :

Δ-(Conn)=(a,b)|x|Conn U Conn|x|(a,b) U (a,b)|x|(a,b) = {(a,c),(a,e)}

The Counting algorithm computes Δ-(Conn) (Δ+ for insertions). Each tuple in Δ is associated with a count value(negative for deletions). In this example, we will have (a,c,-1),(a,e,-1). Combining this with the materialized view One_Stop, we get that the updated view will be

From	To	Count
a	c	1

The counting algorithm performs this procedure for each Materialized View in the system.

a2. Outer Join Views

When Views are defined in terms of Outer Joins, in the following way:

```
CREATE VIEW V  AS
    SELECT X1,X2,...Xn
      FROM R Full Outer  Join  S  on  g(Y1,Y2,..)
```

Where g(Yi..) is a conjunction of predicates.

Tuples can be inserted into or deleted from R or S. This is denoted as

Δ+(R) and Δ-(R).

The outer join can be rewritten as a pair of Left and Right Outer Joins:

q1. SELECT X1,X2,...Xn
FROM Δ(R) **Left Outer Join** S **on** g(Y1,Y2,..)

q2. SELECT X1,X2,...Xn
FROM Rmod **Right Outer Join** Δ(S) **on** g(Y1,Y2,..)

Rmod is R after modifications.

We again explain the algorithm via an example:

Consider R(A,B) and S(A,C), and the instances :

A	**B**
10	15
20	25

A	**C**
10	BB
15	DD

The Full Outer Join is

A	**B**	**A**	**C**
10	15	10	BB
20	25	Null	Null
Null	Null	15	DD

Inserting (25,30) in R will insert (25,30,Null,Null) into the view. If we insert a matching tuple like (15,30), we get (15,30,15,DD). However, note that the algorithm should erase (Null,Null,15,DD). Also note that erasing the last matching tuple (i.e. (15,30)), we should add (Null,Null,15,DD).

In short, q1 computes the effect on View V of changes to R, and q2 does the same with changes to S.

a3. Recursive Views

These views are usually expressed by means of Datalog rules. We will explain an algorithm for view maintenance called **Dred (Deletion and Rederivation).** As its name reveals, this algorithm has two phases. It first deletes all tuples affected by the view expression.
In the example of the counting algorithm, it would delete **both** tuples, (a,c) and (a,e), no matter how many derivations each tuple has. In this way, an overestimation of the deleted derived tuples is obtained. Then, this overestimation is pruned by looking for alternative derivations of the deleted tuples. Thus, (a,c) will be reinserted. Insertion is performed using the view (once partially updated), and the base relations.

B. It is not always possible to maintain a view using only **partial information**. Thus, the algorithms first check if the view can be maintained, and then(if it is possible), do it.
B1.- Queries Independent of Update. Here, an algorithm checks if an update affects the view. If it does not, nothing is done. If it does, then algorithms for maintenance are applied.

B2.- Self Maintenance

A view is called **Self Maintainable** if it can be maintained using only the materialized view and key constraints. This is an important concept in order to apply this to Data Warehousing, because we do not want to scrub into base data to update summary tables.

- We say that a **view is Self Maintainable with respect to a modification type** T **to a base relation R** if the view can be self-maintained for all instances of the database in response to all modifications of type T over R.

In the example considered at the beginning of the section,

$$CAL\_EMP = \pi_{emp\_no}(\sigma_{state= \text{“CA”}}(Projects) |x| Assigned\_to)$$

If we delete a tuple in Assigned_to, like (emp10,pA10,100), we can not delete emp20 from the view without checking if this employee is assigned to another project in California. Thus, the view CAL_EMP is not Self-Maintainable with respect to deletions on Assigned_to. It is obvious that this view is not self-maintainable with respect to insertions into any of the two base relations.

- We say an attribute is **Distinguished in a view V** if it appears in the SELECT clause of an SQL definition of view V.
- An attribute A belonging to a Relation R is **Exposed** in a view V, if A is used in a predicate.

Some important results, which we will not prove here, are :

- An SPJ View taking the join of two or more relations is not self-maintainable with respect to insertions.
- An SPJ view is self- maintainable with respect to deletions in R if the key attributes from each occurrence of R in the join are either included in the view, or equated to a constant in the view definition.
- A left or full-outer join view V defined using two relations R and S, such that the keys of R and S are distinguished, and all exposed attributes of R are distinguished is self- maintainable with respect to all types of modifications in S.

C. Partial Reference.

We are in an intermediate case when we want to maintain a view using just a subset of the base relations, and the view. We will not treat this case here.

We are now ready to get into some topics on how a Data Warehouse can be implemented, and the issues that must be considered.

4.- DATA WAREHOUSE IMPLEMENTATION

In this section, we will review some important techniques proposed for improving decision support performance. They focus on how to efficiently compute aggregations, either by precomputing every aggregate or by selecting an appropriate subset. The intention here is to summarize the most relevant research work in the field. It is worth to remark at this point that, contradicting the relational database

development cycle, Decision Support roots are found in the database industry. However, in the last three years, it deserved growing attention from the database research community . Some of the most important work is covered here.

We start by introducing the **Data Cube Operator**, which generalizes the GROUP BY clause. We present some ideas developed for computing this cube in an efficient way. We then introduce another vision of the problem, in which only a portion of the cube is precomputed, and we show some ideas about how to choose the best possible subset of views. We finish with some topics on summary tables maintenance.

4.1. The Data Cube

As we explained in the previous sections, queries to the data warehouse may take a long time to complete, due to their complexity and the size of the repository. In order to keep response time within acceptable limits, some techniques were developed, mainly making use of view materialization. All of these are based on the multidimensional way in which data is presented to the user. Data is presented as a multidimensional data cube. The sides of the cube represents dimensions interesting for analysis, and the cells are usually called measures. The user seeks information along different portions of this multidimensional data cube. A simple example would be the relation defined by

RegionSales(Product, Region, Sales)

Here, the dimensions of interest are Product and Region. Sales is the measure. An user may ask for total sales by product, by region, or by product and region.

In order to speed up queries we could precompute some aggregates in advance and materialize them, or we can materialize the whole cube. This last approach is the motivation for the Data Cube Operator, introduced in [GBPL97].

A Relational Database is not the best data structure to hold data which is, in nature, multidimensional. Let us look at an instance of the RegionSales relation :

Product	Region	Sales
p1	r1	100
p1	r2	105
p1	r3	100
..	..	..
..	..	..
p3	r1	30
p3	r2	40
p3	r3	50

We see that there is one attribute for each dimension.
Computing aggregates along the two dimensions involves scanning the whole relation.

On the other hand, looking at these data in a true multidimensional way, we would obtain the following matrix :

	r1	r2	r3	Total by Product
p1	100	105	100	305
p2	70	60	40	170
p3	30	40	50	120
Total by Region	200	205	190	595

Clearly, computing aggregates in this way consists only in performing matrix arithmetic. This explains why MOLAP systems, which stores data in special arrays, delivers a better performance.

Suppose now we want to know the total sales per region. The SQL query for this, would look like

```
SELECT Region, sum(Sales)
FROM  RegionSales
GROUP BY Region
```

Let us now add a new dimension, say one stating if the value recorded is actual or a budget. A possible instance of RegionSales may be

Product	Region	Condition	Sales
p1	r1	actual	100
p1	r1	budget	105
p1	r3.	budget	100
..	..	..	..
..	..	..	..
p3	r1	actual	30
p3	r1	budget	40
p3	r3	actual	50

With this representation, it will not be easy to **drill down** or **roll- up** data, this is aggregate data at finer or coarser levels. A solution proposed by C.J.Date would make RegionSales look like

Product	Region	Condition	Sales by Product and Region	Sales by Product and Status	Other aggr.
p1	r1	actual	205	2500	...
p1	r1..	budget	205	3400	...
p1	r3.	budget	...	3400	...
..	..	..	...	...	...
..	..	..	...	...	...
p3	r1	actual	70	4500	...
p3	r1	budget	70	3200	...
p3	r3	actual	50	4500	...

This representation has its drawback in the fact that as the number of attributes grow, so does the number of columns of the table.

The proposed solution was to include a special value called **"ALL"** , to denote the aggregate fields. In this way, a roll-up would be calculated with these SQL statement :

```
SELECT "ALL", "ALL", "ALL", sum(Sales)
FROM  RegionSales

UNION

SELECT Product, "ALL", "ALL", sum(Sales)
FROM  RegionSales
GROUP BY Product

UNION

SELECT Product,Region, "ALL", sum(Sales)
FROM  RegionSales
GROUP BY Product,Region

UNION

SELECT Product,Region,Condition, sum(Sales)
FROM  RegionSales
GROUP BY Product,Region
```

If we want a symmetric aggregation table, we get a **cross-tab**. This is obtained by adding the clauses :

```
SELECT "ALL", Region, "ALL", sum(Sales)
FROM  RegionSales
GROUP BY Region

SELECT "ALL","ALL", Condition, sum(Sales)
FROM  RegionSales
GROUP BY Condition

etc.
```

The aggregates can be precalculated in this fashion. However, a cube of **n** dimensions would require 2^n GROUP BY statements. It would be easier to define a new operator named **CUBE,** which generates the power set of the aggregation columns of the relation. This is, all the possible subsets of aggregates. The semantics of the CUBE operator is the one depicted above. The syntax is something like

```
SELECT Product, Region, Condition, sum(Sales)
FROM  RegionSales
GROUP BY CUBE
```

If the cardinality of th N dimensions are C1,C2,...,Cn the number of tuples to be generated is given by the expression $\prod(Ci + 1)$.

The CUBE operator applied to the RegionSales relation gives

Product	Region	Condition	Sales
p1	r1	actual	100
p1	r1	budget	105
p1	r2	budget	100
..	..	..	..
p1	r1	ALL	205
p1	r2	ALL	300
p1	ALL	ALL	14000
p2	ALL	ALL	15000
...	...	...	...
ALL	ALL	ALL	260000
ALL	ALL	actual	1200000
...	...	...	...
ALL	r10	budget	2900

One interesting point concerns the **size** of the Data Cube, with respect to the size of the underlying relation, because it may or may not justify the materialization of the complete Data Cube. The calculations performed in [GLPB96] give a relation of 2.4 , but it considers that the cube is not sparse. In the example above it means that every product is sold in every region. This is not usually the case, thus, it may not be convenient to materialize the whole cube.

4.2.- Computation of the Data Cube

Computing the data cube could become an extremely hard work unless an adequate strategy is applied. The simplest method, consisting in perform the corresponding Group-by queries and their union, would be unacceptable in real-life applications. Thus, a collection of optimization techniques have recently appeared in the literature, and we comment some of them, below.

We can represent the data cube as a **lattice**, where each node is a group-by of the cube, and each edge goes from node i to node j if j can be computed from i, and the number of group-by attributes of i equals the number of attributes of j, plus one.
A canonical example is given below.

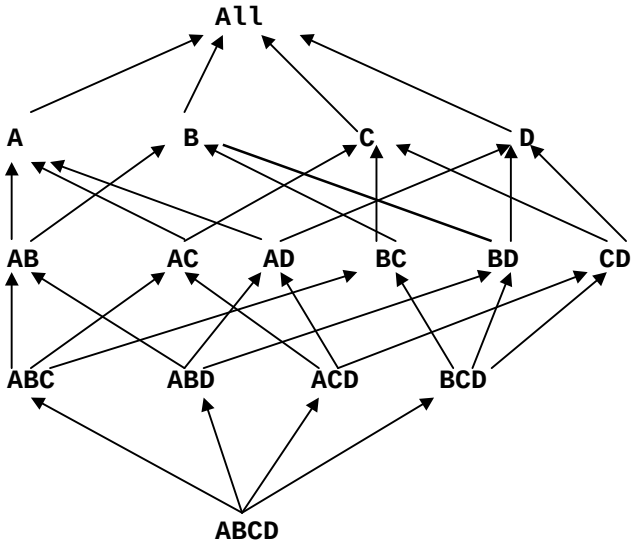


Figure 5

- **Simple optimizations**

The basic methods for group-by computing are based on sorting and hashing[AADGN96]. The simplest optimizations are:

Smallest-parent: Computes each group-by from the smallest previously computed one. In the lattice above, AB can be computed from ABC,ABD, or ABCD. This method chooses the smallest of them.
Cache-results: Intends to cache in memory a group-by from which other ones can be computed.
Amortize-scans: Tries to compute in memory as many group-bys as possible, reducing the amount of table scans.
Share-sorts: Applies only to methods based on sorting, and aims at sharing costs between several group-bys.
Share-partitions: Specific to algorithms based on hashing. If the hash table does not fit in memory, data is partitioned according to a subset of its attributes.

4.2.1.- PipeSort Algorithm.

This algorithm was proposed in [SAG96], and, as the **PipeHash** algorithm, assumes that an estimation of the sizes of each node in the lattice has been calculated.

The PipeSort algorithm gives a global strategy for computing the data cube which includes the first four simple optimization methods specified above. Note that these methods can be contradictory. For instance, Share-sorts would induce to prefer AB to be derived from ABC, while BDA could be its smallest parent.

The algorithm also includes cache-results and amortize-scans, by means of computing nodes with common prefixes in a pipelined fashion. In this way, we could compute ABCD, ABC, AB and A in a pipeline, using just a single scan for all of the nodes.

The **input** of the algorithm is a lattice in which each edge (**e**ij), where i is the parent of j,is labeled with two costs :

S(**e**ij),A(**e**ij)
--------->

S(**e**ij) is the cost of computing j from i, if i is not sorted. A(**e**ij) is the cost of computing j from i if i is already sorted.

The **output** is a subgraph of the input lattice, where there is only one edge between i and j. If j is a prefix of i, it means that j will be computed from i at cost A(**e**ij). If not, i should be sorted first, and cost will be S(**e**ij).

The goal of the algorithm is to find an output graph with minimum sum of edge costs. This is achieved by applying to every pair of levels k and k+1 in the input lattice(assume k gives the number of group-by attributes at each level), an algorithm that finds a **minimum cost matching**. This works in the following way:
Consider a pair k, k+1 of levels, beginning from 0, where nodes in level k are computed from nodes in level k+1. Transform level k+1 by making k copies of each of its group-by nodes. There will k+1 edges going out from each node at level k+1, but only one of them at cost A(**e**ij). Note this will be a bipartite graph. A minimum cost matching will be produce a graph with at most one edge joining a node in level k to a node in level k+1, such that the total sum of the

weights will be minimum. As an example, consider levels 1 and 2 of the example above. We have added a copy of each node at level 2.

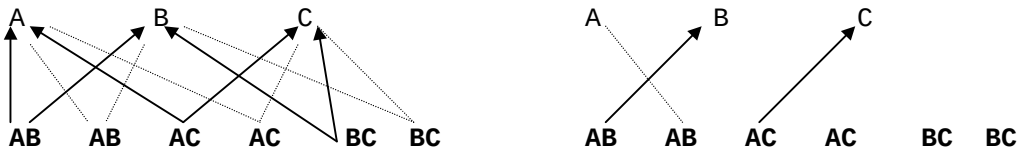


Figure 6

In this example, B will be computed from BA (ordered), while C will be computed from CA, both at a cost A(eij). A will be computed from AB, but at cost S(eij).

The algorithm performs this matching N times (N is the number of group-by attributes) generating an evaluation plan. Note it may be possible that, although at each level the total cost is minimum, some nodes could have to be sorted more than once. The output lattice gives a sorting order for each node. As a result, the PipeSort algorithm induces an evaluation strategy : In every chain such that a node in level k is a prefix of node in level k+1 (in the output graph), all group-bys can be computed in a pipeline.

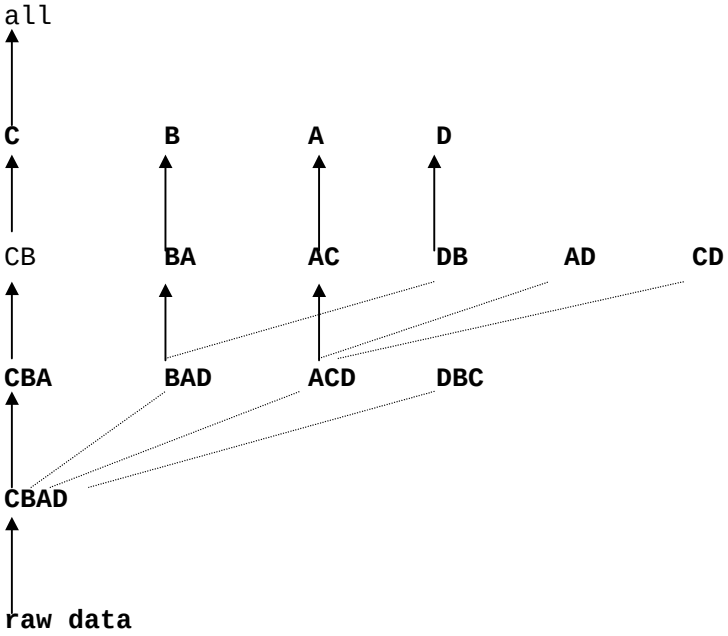


figure 7

In the above figure, we show a minimum cost matching. The minimum cost sort plan will first sort in CBAD order, and compute CBA, CB, C and ALL group bys in a pipelined fashion. Then, we should sort in order BADC, and do the same with group-bys BAD, BA and A. We continue in the same way with ACDB and DBCA.

4.2.2.- PipeHash Algorithm

This algorithm incorporates optimizations cache-results, amortize - scans and smallest parent. If memory is not enough for all hash tables to fit, data must be partitioned, which leads to optimization share-partitions. The pipeHash algorithm[SAG96], contrary to pipeSort, is based on the smallest parent optimization. The input is the lattice formerly shown, with the nodes labeled with the estimated size of the group-by. The output is a minimum spanning tree (MST). In this tree, each edge goes from the smallest parent to the node which is computed from it. The next figure shows a minimum spanning tree. Each node is labeled with the size of the corresponding group-by.

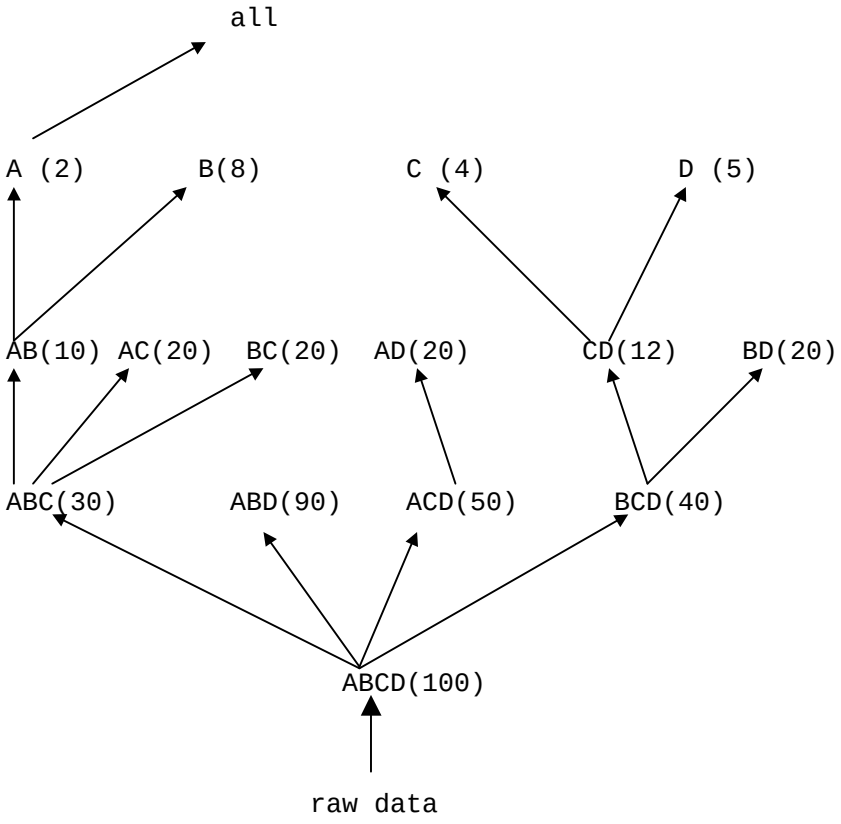


figure 8

As in general it is not possible to compute all the nodes in a MST together, the tree is decomposed in several subtrees. The idea is to choose a partitioning attribute from the root of the MST T (in case memory is not enough to compute the entire tree), and obtain a subtree Ts, the nodes of which contain the partitioning attribute.
The subtree for the former example(with partitioning attribute A), is shown in figure 9.

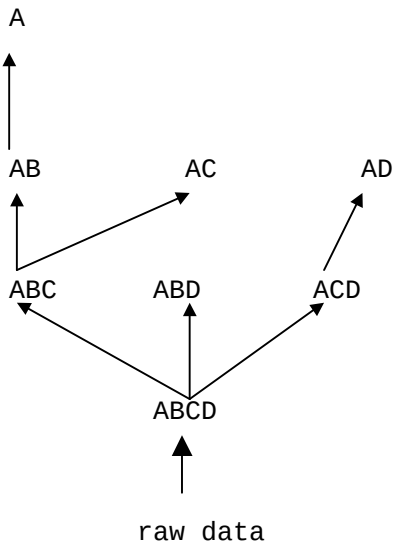


figure 9

The same process is repeated with the subtrees in the remaining forest T-Ts.
An instance for the current example is given below(consider data is accessed via a hash table):
The partitioning attribute is A.

raw data

```
a1 b1 c1 d1 30
a1 b1 c1 d1 10
a1 b2 c2 d2 20
a1 b2 c4 d1 30
a1 b3 c2 d3 40
a2...
....
```

1st partition ===>

ABCD

```
a1 b1 c1 d1 40
a1 b2 c2 d2 20
a1 b2 c4 d1 30
a1 b3 c2 d3 40
```

==>

ABC

```
a1 b1 c1 40
a1 b2 c2 20
a1 b2 c4 30
a1 b3 c2 40
```

ABD

```
a1 b1 d1 40
a1 b2 d2 20
a1 b2 d1 30
a1 b3 d3 40
```

ACD

```
a1 c1 d1 40
a1 c2 d2 20
a1 c4 d1 30
a1 c2 d3 40
```

The computation continues for this partition, by storing ABCD and ABD on disk, and computing AD from ACD, storing both, computing AB and AC from ABC, storing ABC and AC, and finally computing A from AB. The same is performed for the other subtrees.

Experimental evaluations showed that, in general, pipeSort does better than pipeHash, mostly because the hash table overhead. However, for non-sparse data, pipeHash performs better than pipeSort.

4.2.3.- Overlap Sort

The Overlap Sort Method is presented in [DAN96]. Is, as PipeSort, based

on sorting. Moreover, the Overlap Method is also based on a slight variation of the smallest-parent optimization. This variation computes group-bys with k grouping attributes, from "parents" having k+1 attributes. In what follows, we will call each node in the lattice as a **cuboid.** The cuboid having all attributes will be called the **Base Cuboid**. The method attempts to compute cuboids in an overlapped fashion.

First, we shall give some definitions :

- **Sorted Runs :**

Consider a cuboid {A1.....Aj}. The cuboid is sorted in the order given by A1...Aj. Consider now another cuboid, S = (A1,...,A_{l-1},A_{l+1} ,..,Aj). S is computed from B = (A1,...,Aj). A Sorted Run R of S in B is defined as π A1,...,A_{l-1},A_{l+1} ,..,Aj (Q), where Q is a maximal sequence of tuples t of B such that for each tuple in Q, the first l columns have the same value. As an example, let B = [(a,1,2), (a,1,3), (a,2,2), (b,1,3), (b,3,2), (c,3,1)], and S, the cuboid on its first and third attribute. Thus, S = [(a,2), (a,3), (b,3), (b,2), (c,1)]. The sorted runs for S in B are : [(a,2), (a,3)], [(a,2)], [(b,3)], [(b,2)], [(c,1)].

- **Partitions**

Taking into account what was said for sorted runs, If B and S have a common prefix A1,...,A_{l-1} , a partition of S in B is the **Union** of sorted runs such that the first l-1 columns of all the tuples of the sorted runs have the same value. In the former example, the partitions would be [(a,2), (a,3)], [(b,3),(b,2)], [(c,1)].

The importance of the above definitions lies on the fact that partitions can be computed independently from each other, thus, a partition is a unit of computation.

The Method.

The algorithm first sorts the base cuboid. All other cuboids can be computed without any further sorting.

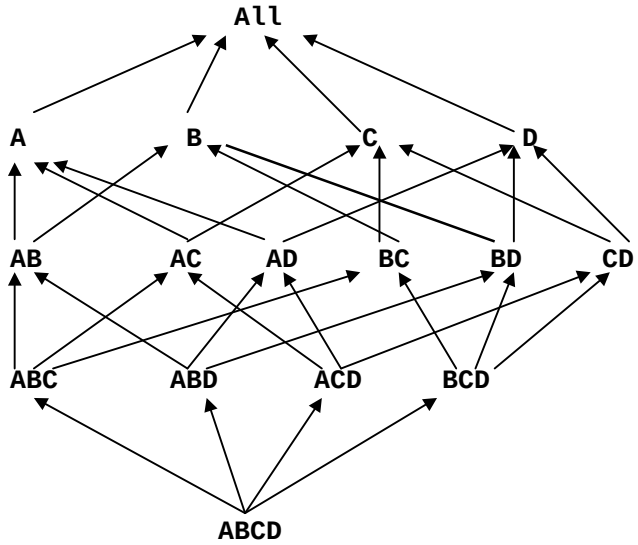


Figure 10

In figure 10, if the base cuboid is sorted in ABCD order, the other cuboids will be sorted in the same order.

It is clear that if enough memory is available, the whole cube can be computed in one scan of the relation, but this is not often the case. Then, the method intention is to use just the necessary memory to compute a partition. The partitions, as we explained, are units of computation, so they can be calculated independently from each other. Moreover, in a way similar to the one mentioned in 4.2.1 and 4.2.2, once a partition is computed, we can pipeline it for the computation of the descendant cuboids(or saved to disk). In this way, we free memory for the next partition. The sizes of the partitions depends on the sorting order of the base cuboid. In this way, partition size for ABC will be 1, (computed from ABCD), while partition size for ABD will be greater than 1(but at most, it will be given by the number of different values of D).

The algorithm involves three steps :

1. Choose a parent to compute a cuboid.
2. Compute the cuboid from its parent.
3. Choose a set of cuboids for overlapped computation.

The first step is straightforward, if we try to minimize the size of the partitions: we just need to follow the sorting order of the base cuboid. This is based on what we explained above: partition size of AC computed from ACD is smaller than if we compute it from ABC. The sizes of the partitions can be estimated from the estimated sizes of the cuboids. We will talk later about methods to perform these estimations. We then turn the graph of figure 10 into a rooted DAG. This is showed in figure 11.

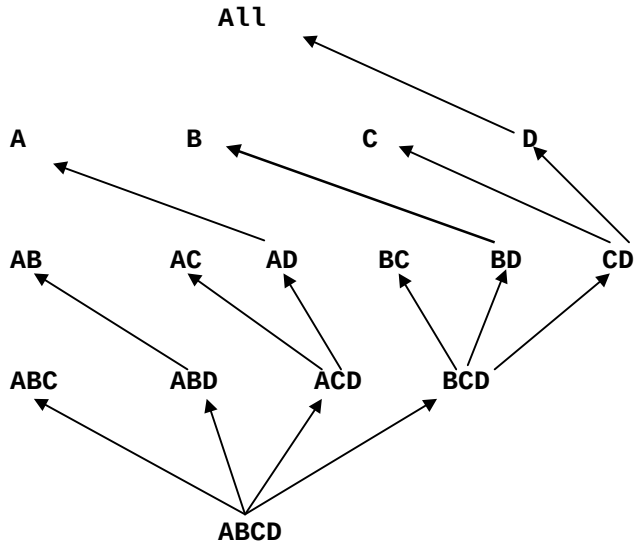


Figure 11

An estimation of the partition size can be obtained through the formula

$$\frac{\text{Size of cuboid } (A1,\ldots,A_{l-1},A_{l+1},..,Aj)}{\text{Size of cuboid } (A1,\ldots,A_{l-1})}$$

If we note that any cuboid can be computed from another one having one

more attribute, say l, then, an heuristic would be to choose the cuboid with the largest l to compute S, among all the possible ones. This has been applied to the graph of figure 10. For instance, for computing AB, we chose ABD instead of ABC. Then, the sizes of the partitions increases from left to right in the tree for the children of any node.

Now, we turn into the computation of a cuboid from its parent. As we said, there would be no problem if there is enough memory to fit the largest partition. When this is not the case, we compute the sorted runs, write them to disk, and finally perform a merge-sort. Proceeding in this fashion, there is only one page needed for writing a sorted run to disk. Then, we have to states of the algorithm : **Partition** or **Sorted Run**, according to the amount of memory available. Each time a new partition is launched, we save the old partition to disk, thus freeing memory. We proceed in an analogous way for sorted runs. To decide the state each cuboid belongs to, we must have an estimate of partition sizes. Then, we can label each node in the tree with it.

It is important to notice that when a cuboid is in a partition state, its tuples can be pipelined, which is not the case for the sorted run state. In the first case, all descendants of a cuboid can be computed in the same pass. In the case of sorted run cuboid, its descendants can not be computed until the complete cuboid is.

As an example, consider the base cuboid ABCD and its instance :

ABCD

a1	b1	c1	d1	100
a1	b1	c1	d2	100
a1	b2	c1	d1	100
a2	b2	c1	d2	100
a3	b2	c2	d1	100
a3	b3	c2	d1	100

Suppose ABC, ABD and ACD belong to partition state, whereas BCD belongs to sorted run state.

When we read the first two tuples, we obtain the following instances of the cuboids :

ABC				**ABD**				**ACD**				**BCD**			
a1	b1	c1	200	a1	b1	d1	100	a1	c1	d1	100	b1	c1	d1	100
				a1	b1	d2	100	a1	c1	d2	100	b1	c1	d2	100

Note that, as ABC are the same, they are summed and collapsed into one tuple.

When the third tuple is read, we store partition ABC (size of partition equals 1), and start a new partition. The same happens with ABD (ABD is partitioned according to the values of AB). Current partition in ACD and sorted run on BCD remain valid. The instance will be :

ABC				**ABD**				**ACD**				**BCD**			
a1	b2	c1	100	a1	b2	d1	100	a1	c1	d1	200	b1	c1	d1	100
								a1	c1	d2	100	b1	c1	d2	100
												b2	c1	d1	100

As AB is computed from ABD, the stored partition can be pipelined, increasing the amount of overlapped calculation.

Algorithm continues in the same way. The sorted run for BCD ends when tuple <a2,b2,c1,d2> is read (we get a different value for A). When all tuples are read, all sorted runs are merged, and only then children of BCD can be calculated.

There is not reference about how the Overlap Algorithm performs, compared to Pipesort and Pipehash.

4.2.4.- Cube size estimation.

From what was explained above, we can infer that it would be very useful if we could predict the sizes of the different aggregates, either if we are trying to know in advance the amount of disk space needed to store the data cube, or as a tool to be used in the algorithms which compute it. [SDNR96] gives three methods which we will explain in what follows. The first of these methods is purely analytical. The second is based on sampling, and the last one on probabilistic counting. All the methods consider the existence of hierarchies along the dimensions of the cube. Other fact to consider is that data distribution influences the size of the data cube.

4.2.4.1.- The Analytical Algorithm.

This method is based on a result found by Feller in 1957, stating that choosing r elements (which we can assume are the tuples in a relation) randomly from a set of n elements (which are all the different values a set of attributes can take), the expected number of distinct elements obtained is $n - n * (1 - 1/n)^r$. It assumes data is uniformly distributed. If it turns out not to be the case, and data presents some skew, we will be overestimating the size of the data cube.

For instance, let us suppose R = (prod_id, store_id, region_id). If we want to estimate the size of a group by on (store_id, region_id), we should know the number of different values of each attribute. Then, n would be |store_id|*|region_id|, and r the number of tuples in R.

Notice that we should estimate the size of each possible group by in order to estimate the total size of the cube. The total number of group bys to be computed is $\Pi_{i=1,n}(hi + 1)$, where hi is the size of the hierarchies involved. We see that hierarchies increases the number of group bys.

The obvious drawback of the algorithm is that it does not consult the database. Its main advantage, its simplicity and performance. It can be used when we know that data is uniformly distributed.

4.2.4.2.- The sampling-based algorithm.

The idea is to take a random subset of the database and compute the cube on this subset. Let D be the database, S the sample, and CUBE(s) the size of the cube computed from S, the estimate of the size of the cube will be CUBE(s)*|D| / |S|.

This method is simple and fast, and the reported results are good for a considerable number of data sets.

4.2.4.3.- An algorithm based on probabilistic counting.

The main idea is to count the number of distinct elements in a multiset. Again, this algorithm estimates the sizes of the group bys in a cube, but this time at the cost of scanning the database once. However, this is cheaper than actually computing the cube, and the authors provide a bound of the method´s error.

The method proceeds as follows:

Let **y** be an integer of L bits. Also let ρ(y) be a function representing the position of the least significant 1-bit in the binary representation of **y,** hash() a hashing function transforming records into integers ranging between 0 and L, and BITMAP[0..L-1] a bit vector. If M is a multiset whose cardinality we want to find, we scan M and set BITMAP[i]=1 when i=ρ(hash(x)), for each x in M. In this way, call A the position of the leftmost zero in the bitmap. 2^A will be the estimate of n (the size of M).

It is clear that the error will be within a factor of 2 from the actual size, so a way to improve this method would be to use m hash functions and m BITMAP vectors, taking the average of the Ai´s obtained in this fashion. A variant to avoid computing m bitmaps consists in distribute each record into m lots via the function

α = hash(x) mod m

updating only the corresponding BITMAP vector, not everyone. Then, proceeding as before, the estimate of n will be

n= m* 2^A / φ , where φ = 0.77351.

The details can be found in [SDNR96]

Finally, in order to estimate the size of the cube in one scan, the algorithm is :

```
for  each tuple T in the database do
    for each combination C of hierarchies do
        T´ := Π_C (T)
        bitset(C, α(T´),bit(T´))
count:=0
for each combination C of hierarchies do
  Add  the estimation for C to count
```

4.2.4.4.- Results

The analytical algorithm is close to the actual size when data is uniformly distributed. But if data is skewed, the size of the cube decreases and the algorithm overestimate it.

The sampling algorithm does not see enough duplicates, so it also overestimate the size of the cube.

The algorithm based on probabilistic counting has, as we commented above, a bound in its error.

4.3.- Algorithms which does not compute the entire Data Cube

In the previous paragraphs we explained how to compute the entire data cube. However, this is not the only choice vendors had made in order to

implement decision support systems. Generally speaking, three alternatives exist to implement a Data Warehouse : materialize the whole data cube, materialize nothing, or materialize a selected part of the cube. Essbase had chosen the first one, while Metacube preferred the latter, and Business Objects the second one.

Materializing the whole cube has the drawback of the storage space required. We saw in section 4.1 that the number of tuples with an “ALL” value can be large compared to the rest of the tuples in the data cube. Then, materializing only a part of the cube can be an interesting alternative which provides good performance at an acceptable cost of storage. In [HRU96], a technique is presented in order to help in the task of deciding which elements of the data cube materialize. It gives a framework which allows us to answer three typical questions:

1. How many views must we materialize to get reasonable performance ?
2. Given storage space S, what views should we materialize in order to minimize the average query cost?
3. If we can assume an X% performance degradation from a fully materialized data cube, how much space do we save?.

The mentioned work propose a lattice framework as follows :

Suppose we have a 3-D data cube (in this case, the same as presented in the TPC-D benchmark)

(part, supplier, customer)

We can consider it as a summary table.

We can built several group bys from it, and we can represent these group bys as a lattice (in an analogous way to what we did in section 4.2.)

Sizes(in millions of tuples)

psc : 6

pc : 6

ps : 0.8

sc : 6

p : 0.2

s : 0.01

c : 0.1

() : 1

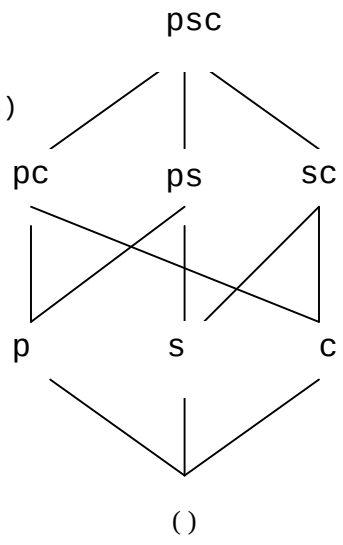


figure 12

As an example, notice that, as view psc is materialized(we always do it

with the root view in order to avoid looking into the raw data), and its size is 6 million tuples, it is of no use to materialize view pc because the query that groups by part and customer will require to scan a 6 million tuples relation. Also compare the cost of answering a query that groups by supplier using view s(0.01 million rows scan), instead of doing it from view sc (6 million rows scan).

If we consider each view in the lattice above as a query, say Qi, that lattice implies a relation between queries in it. Let Qi and Qj be two queries located at different levels. If Qi can be answered using the results of Qj (v.g. p from ps above) we say that there is a dependence relation $Qi \leq Qj$.

For instance,
$part \leq part, customer$, but
$part \not\leq supplier$

The resulting lattice is denoted $<L, \leq>$ and called a **dependency lattice.**
The lattice presented above, is enhanced with the possibility to represent hierarchies, becoming more complex than a hypercube lattice as the one of figure 12. This is important as hierarchies are the basis for drill-down and roll-up operations. Hierarchies also impose a dependence relation between queries, as the one depicted in the following example for the time dimension:

$year \leq month \leq day$ and

$week \leq day$

Then, if we have a grouping over month, we can use it to compute the grouping over year.

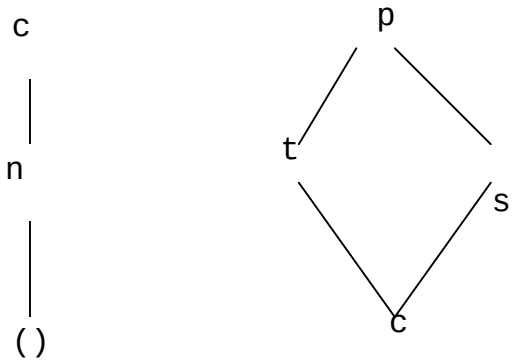
The lattice introduced in [HRU96] allows the representation of two types of query dependencies:

1. Those caused by the interaction between different dimensions.
2. Dependencies within a dimension caused by hierarchies.

If we have different hierarchies along n dimensions, we can obtain a lattice, whose views are of the form (a1,..,an), where each ai is a point in the hierarchy for the ith dimension, as the **direct product** of the dimensional lattices. These views accomplish the relation

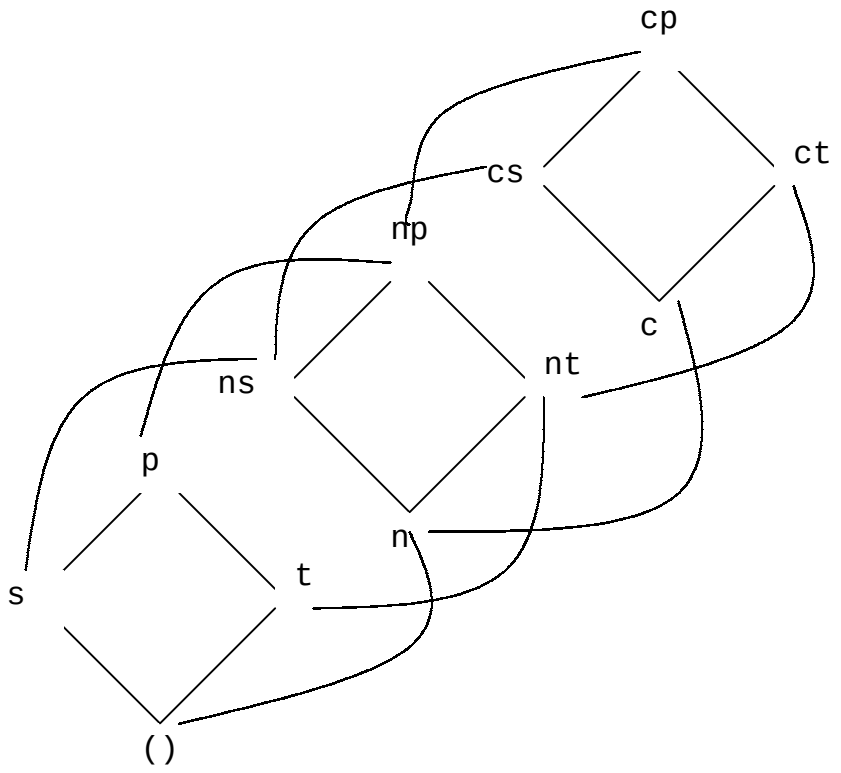
$$(a1,a2..,an) \leq (b1,..bn)$$

As an example, let us suppose two hierarchies with roots Customer and Product respectively.



n, t, and s represent country, type and size respectively.

If we perform the direct product of these hierarchies, we obtain the following lattice :



Notice that in each edge ai ≤ bi is satisfied. For instance, s ≤ p, t≤ p, n ≤ ns , etc.

We can conclude that the main advantages of the lattice framework are :

1. It integrates dependencies caused by hierarchies with the ones caused by dimensional interaction.
2. It Gives an order of materialization. This can be achieved by means of the partial order induced by the ≤ operator. Given a set of views to materialize, say S = {Q1,...Qn}, if we perform a topological sort of S based on ≤, we can materialize them in the resulting order. In this way, we would be able to use a previously materialized view Qi to materialize another view Qj (except from the views with no ancestor, which must be materialized from the raw data).

In order to give an algorithm for choosing efficiently a subset of views to materialize, a cost model must be given. In [HRU96], a linear cost model is assumed, with the following characteristics :

1. The cost of answering a view Q from a materialized view Qa, is the number of rows in Qa.
2. Full-view and single-cell queries behave in a similar way.
3. All queries are identical to some view in the corresponding lattice.

As in the algorithms which compute the whole data cube, we must predict view sizes. The methods presented in the previous section may be used for this task.

We will now present an algorithm to optimize the data cube lattice. The goal will be to minimize the time taken to evaluate a view, constrained to materialize a fixed number of views, regardless of the space available. As this problem is NP-complete, the algorithm(a “greedy” one) uses an heuristic which consists in selecting a sequence of views such that each choice in this sequence is the best, given what was selected before. The algorithm is :

INPUT : A lattice, in which we know the expected number of rows for each view.

OUTPUT : The set of views to materialize.

We call C(v) the cost of view v, and k the number of views to materialize. If S is a set of already materialized views(initially only the top view is in S), the **Benefit of view v relative to S, B(v,S)**, is defined as:

1. For each $w \leq v$, Bw is

   a. Let u be the view of least cost such that $w \leq u$.
   b. If C(v)< C(u), Bw = C(v) – C(u). Otherwise, Bw = 0.

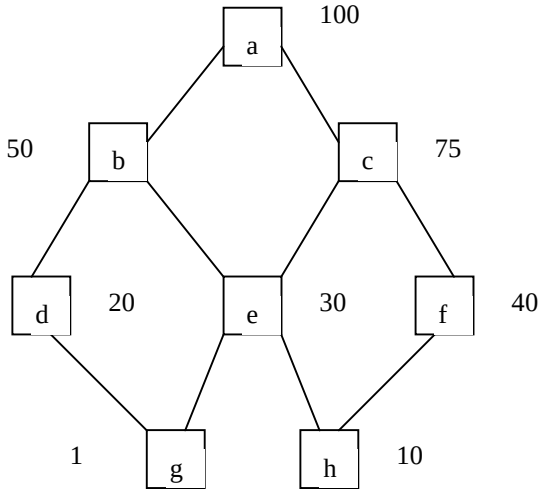
2. $B(v,S) = \Sigma_{w \leq v}$ Bw

Shortly, we compute the benefit of materializing a view v considering how this can improve cost of evaluating the views in the lattice.

Finally, the algorithm is :

S = {top view}
For i=1 to k **do**
 select view v not in S such that B(v,S) is maximized
 S = S union {v}
end
S is the selection of vies to materialize.

The following example will clarify things. In the input lattice, the nodes are labeled with symbolic costs. We can choose 3 views to materialize.



Let us show the how to select the first view to materialize.
We first calculate $B(b,S) = \Sigma_{w \leq b} Bw$

For each view v covered by b, we compute C(a) – C(v), because a is the only materialized view when the algorithm begins. Then,

B(b,S) = 50 + 50 + 50 + 50 + 50 = 250.

Each term is the benefit produced by the materialization of view b on the calculation of views b, d, e, g, and h.

In an analogous way,

$B(d, S) = \Sigma_{w \leq d} Bw = 80 + 80 = 160$.

Each term corresponds to the benefits on d itself and g.

If we continue in this fashion, we will find that b is the view to materialize.

In the second step,

$B(c,S) = \Sigma_{w \leq c} Bw = 25 + 25$ (corresponding to c itself and f).

$B(f,S) = \Sigma_{w \leq f} Bw = 60 + 10 = 70$ (f and h).

We will eventually choose f.

The three views are a, b and f, with a total benefit of 380 (250 + 70 + 60 respectively).

It can be proved that the benefit of the greedy algorithm is at least 63% of the benefit of the optimal algorithm.

4.4.- Maintaining the Data Cube

In the previous sections, we have seen how to implement the data cube, and how we can improve query performance by the materialization of the different aggregates. These materialized aggregate views are called **summary tables.** Besides, in section 3 we introduced some concepts about materialized views. In this section we will comment some ideas about how this summary tables can be maintained with minimum access to the raw(base) data, and with maximum data availability.

The problem can be stated in the following way:

- As data at the sources is added or updated, the summary tables which depend on these data must be also updated. Two options arise : to recompute the summary tables from scratch or to apply incremental view maintenance techniques to avoid such recomputation.
- While summary tables are maintained, they remain unavailable to the data warehouse users. This forces the necessity of reducing the time invested for their updating, because maintenance must be performed during the night.

We will focus on the work referred in [MQM97], which presents an algorithm for efficiently maintaining summary tables, taking into account the two problems formerly described. Other techniques have been

reported in order to attack the second problem, as the 2VNL algorithm introduced in [QW97]. This work proposes to maintain two versions of the data warehouse, in order to have it available 24 hours a day and 365 days a year.

The algorithm proposed in [MQM97] applies to distributive aggregate functions. Aggregate functions can be :

distributive : can be computed partitioning their input into disjoint sets. Examples of distributive functions are COUNT, SUM, MAX, MIN. However, COUNT(DISTINCT) is not distributive. For instance, if we have the set (3,5,8,3,3,4,4,7), partitioning it into (3,3,4), (5,8,4), (7,3,8), gives us a COUNT(DISTINCT) of 8, while the answer should be 5.
algebraic : can be expressed as a scalar function of the distributive functions. AVG = SUM()/COUNT() is an example.
holistic : can not be computed by dividing into parts. Median is an example of this kind of function.

In section 3 we defined a self-maintainable view[LW95]. In an analogous way we define an aggregate function as self-maintainable if the new value of the function can be computed solely from the old ones and from changes to the base data.

Aggregate functions must be distributive in order to be self-maintainable. All aggregate functions are self-maintainable with respect to insertions, but not to deletions. In fact, MAX and MIN are not, and can not be made, self-maintainable with respect to deletions.

The algorithm proposed has two main processes, one being the **propagate phase**, and the other, the **refresh phase**. The main advantage of this approach is that **propagate** can be run without reducing warehouse availability. Only the **refresh** phase must occur within the updating window. The algorithm can also consider the lattice presented in section 4.3.

The idea is to create a summary-delta table in which store the net changes to the summary table due to changes in the source data. This is performed in the **propagate** phase. Then, during the **refresh** phase, this changes are applied to the summary table.

We will explain the algorithm via an example.

Let us suppose a fact table and two dimension tables.

sales(store_id,item_id,date,qty,price)
stores(store_id,city,region)
items(item_id,name,category)

Consider a view SID_sales as

```
CREATE VIEW SID_sales AS
 SELECT store_id,item_id,date,COUNT(*)as TotalCount,sum(qty) as Totalqty
 FROM sales
 GROUP BY store_id,item_id,date
```

The COUNT(*) is added in order to maintain views in presence of deletions.

Suppose sales_ins and sales_del be tables storing insertions and deletions to the source data. In the **propagate phase**, a new table(view) is created, where the net changes to the summary tables are stored. This

is called a **summary_delta table**(prefixed as sd).

```
CREATE VIEW sd_SID_sales (store_id,item_id,date,sd_count,sd_qty)
AS
SELECT store_id,item_id,date,SUM(_count) AS sd_count,SUM(_qty) AS sd_qty
FROM (( SELECT store_id,item_id,date,1 as _count,qty AS _qty
        FROM sales_ins)
        UNION ALL
      (SELECT store_id,item_id,date,-1 as _count,-qty AS _qty
        FROM sales_del))
GROUP BY store_id,item_id,date
```

Notice that the summary table SID_sales does not have to be locked during the **propagate phase**.

During the **refresh phase**, the net changes, stored in the summary delta table, are applied to the summary table itself. The only case in which base data must be accessed is when MAX and MIN aggregate functions are involved, and a deletion occurs.
The **refresh** algorithm is roughly described below :

```
For each tuple t in sd_SID_sales do
  if not exists(SELECT * FROM SID_sales d
                WHERE t.store_id = d.store_id AND
                      t.item_id = d.item_id AND
                      t.date = d.date)
  insert t into SID_sales
  else
  (if t.sd_count + d.TotalCount = 0 /* condition A
      delete t from SID_sales
      else
      d.TotalCount+= t.sd_count
      d.Totalqty+= t.sd_qty)
```

It should be remarked that, if the aggregate function is MAX or MIN, and condition **A** is not met, the **refresh** algorithm must recompute the aggregates from the base data. If **t** is deleted or not found, no lookup into the raw data is required.

Another interesting feature of the algorithm is that summary tables can be expressed in terms of other summary tables. For example :

```
CREATE VIEW sd_sCD_sales  AS
SELECT city,date,SUM(sd_count) SUM(sd_qty)
FROM sd_SID_sales s1, stores s2
WHERE s1.store_id = s2.store_id
GROUP BY city,date
```

4.5.- Summary of this section

We conclude this section summarizing what we have presented : the data cube concept, the cube operator, and, related to this, how the data cube can be computed and maintained. Alternative approaches intending to improve query performance were also presented: total and partial materialization of the cube. We also commented how the sizes of the aggregates can be predicted, which is applied in several algorithms.

5.- DATA ACCESS - INDEXING - QUERY PROCESSING STRATEGIES

As we have seen, queries on an OLAP system are of a very different nature than the ones which an OLTP system must handle. Data models which focus on dimensional modeling attempt to capture these new characteristics. As a consequence, vendors and researchers began to think about better ways to solve queries involving aggregations, rankings, etc., on databases of several Gigabytes of data. One of such ways has been handling parallelism. Other, which we will review here, is indexing. New index structures were developed, the most relevant being the so-called bitmap indexes[IOL]. Recall that whereas in an OLTP system we expect to get results of a limited number of records, this is not the case on OLAP. Moreover, as OLTP databases and Data warehouses lie on different systems, indexing limitations do not apply to the latter, so we are not limited by the amount of columns to index. As a result, Data Warehouses will, in general, be indexed with several high selectivity indexes.

For instance, if we have the following query :

```
SELECT store_id, sum(sales)
  FROM Customer c, Sales s
   WHERE c.cust_id = s.cust_id AND
         c.state = “NY” AND c.sex = “M” AND c.education= “Univ”
   GROUP BY store_id;
```

In an OLTP system we should not have an index on c.sex due to its high selectivity. Also, having indexes on several columns would lead to problems during updates. So, a query like this would probably be solved performing a selection by one index. It is not difficult to realize that the response time would then be unacceptable for most DSS systems, because of the large amount of rows to be scanned.
In the query above, the query processor must perform a selection on the Customer table, which will result in a number of tuples being retrieved according to the selectivity of attributes state, sex and education, and then a join with the Sales table.

Modern indexing techniques and query processing strategies attempt to deal with this problem. We will review some of them.

bitmapped indexes

In a traditional B-tree index, we find in the leaves, a value and a list of positions of rows satisfying that value. In some cases, we can do better than that, if, instead of a list of such positions, we place a vector of bits in the following way:

Let us assign a number to each row of a table(vg. the Customer one). Suppose we wish to build an index on attribute education. Further, say there are five possible values for this attribute. Imagine we have a bit vector of length N (the number of rows in Customer). In each position **i** of this vector, we place a 1 or a 0 if the Customer in row i has universitary education or not, respectively. If we create a vector for each possible value of the attribute, and place such vector at the leaves of the B-tree, we have a **bitmapped index**. Consider the following bitmap :

```
10001000100001........1
```

This tells us that customers in rows 0,4,8,13...and N, have universitary education.

Here is how to estimate the size of the index :If table Customer has 100.000 rows, and we have 5 possible values for attribute education, a bitmapped index on education(just considering leaf nodes) will occupy 100.000 x 5 /8 = 0.06 Mb. A traditional B-tree index like the one described at the beginning of this section, will occupy 100.000 x 4 = 0.4 Mb. This is not relevant in terms of space for this example, but it would be if we index on a table of several Gigabytes(like the Sales table).

We deduce from the last paragraph that space required by a bitmapped index is proportional to the number of values in the index, while space required by traditional indexes is independent of this number. So, for DSS systems, which encourages indexing on high selectivity attributes, this seems to work fine.

But the main advantage of bitmapped indexing is the high performance achieved in the querying process. When performing an AND, OR or NOT operation, the systems will just be doing a bit comparison. As an example, in the above WHERE clause we had :

c.sex = “M” AND c.education= “Univ”.

The bit vectors for each value are

10001000100001.....1 and

10010100010010.....0

A simple “C” program will can generate another bit vector which will hold the result of the AND operation, performed bit by bit.

1000000000000.....0

This operation is performed at a very low CPU cost by any system. On the other hand, comparing lists is a hard work, requiring the use of cursors.

The only exception to what was said above is the case when the lists are small, or, equivalently, the bit vectors are **sparse**, this is, when there are few value “1” bits. An alternative is to perform **bit vector compression .** This can be done by changing the way in which data is represented, shifting to lists when the bit vectors become extremely sparse. Systems should be able to deal with the different forms of bitmaps.

Join Indexing

We know that Join is the most expensive operation within a DBMS. We will not review the well-known strategies for join processing, like hash-join, sort-merge join, etc, but we will focus on strategies suitable for Decision support, particularly, the use of join indexes and star joins. These strategies take advantage of the Star Schema design of the Data Warehouse, in which a central table (the Fact table) is related to the dimension tables by foreign keys. The joins that will be required by the queries will make use of these foreign keys.

Suppose we have the query:

```
SELECT store_id, sum(sales)
FROM Customer c, Sales s
WHERE c.cust_id = s.cust_id AND c.state = “NY”
GROUP BY store_id;
```

We can create a **Foreign Column Join Index (FCJ)**, on table Sales, but on attribute c.state. This seems to be strange, because state does not belong to Sales. Actually, what happens is that we have an index pointing to the tuples in Sales which stores sales to customers in New York. This is done by first joining both tables by cust_id, and creating one bitmap on sales for each state.

SALES

```
c1 p1  ..... 100  0
c2 p1 .......300  1
c1 p3 .......340  2
c4 ..........100  3
c3 ...........50  4
....
.....
c10 .........200  10000000
```

CUSTOMER

```
c1 .....NY
c2 .....MA
c3 .....CA
c4 .....NY
...
...
```

The FCJ on sales would look like this

```
CA  0000100.....0
MA  0100000.....0
NY  1001000.....0
..
....
```

Each bitmap will have N bits, N being the number of tuples in the Sales table.

In case of having more conditions, we can create more FCJ indexes one for each condition. Performing an AND operation filters the rows in the Sales table which will surely not contribute to the answer.

We can generalize this by not restricting the FCJ to only one table. It is easy to see that this works also when conditions are placed on different dimension tables. This has the advantage of bringing flexibility. Changing or adding conditions does not require a lot of effort. This does not occur with the next Join indexing strategy we will mention.

The syntax for the creation of the FCJ index for the query presented is, in the INFORMIX system :

```
CREATE gk INDEX custsatesales
ON Sales
(Select as key c.state
 from Sales S, Customer C
 WHERE S.cust_id = C.cust_id)
```

Some systems also allow **Multitable Join Indexes (MTJ)**, in case the need of joining more than two tables arises. The idea is the same as the one explained before, that is, to filter values from the fact table. But in this case, a multicolumn index is created, resulting in lack of flexibility and other problems.

Suppose we have the Sales Fact table, two dimension tables, say Time and Customer, and the query.

```
SELECT store_id, sum(sales)
FROM Customer c, Sales s, Time t
WHERE c.cust_id = s.cust_id AND s.time_id = t.time_id
      AND c.state = "NY"  AND t.quarter = "3-96"
GROUP BY store_id;
```

To build a MTJ index on sales, we must perform the join of the three tables and create an index of the form (c.state,t.quarter), with an entry for each pair of values in the Sales table.

To create this index, we must do the following :

```
CREATE gk INDEX custtimesales
ON Sales
(Select as key c.state,t.quarter
 from Sales S, Customer C, Time T
 WHERE S.cust_id = C.cust_id AND S.time_id = T.time_id)
```

Of course, systems implementing this kinds of indexing strategies should be aware of the existence of these indexes when processing a query.

Star Joins

Star Join is a technique which improves join performance without creating join indexes, assuming there is an index on each of the attributes involved in the restrictions.

For the multitable query presented above, the method would first select all cust_id values for customers in "NY", placing them into a temporary table T1. Then, it would select all t.day in the third quarter of 1996 and place them in another table T2, and finally form the cartesian product T1 x T2 in order to join it to a preexistent (cust_id, day) index on Sales.

The main drawback of this technique arises in case the fact table is sparse, which is quite frequent for large organizations. In this case, we spare resources in computing the cartesian product, to form rows that will not join. The techniques described before do a better job in such situation.

There are some other techniques, but they are variations of the ones presented in this survey. The interested reader can look in the bibliography section.

6.-SUMMARY AND CONCLUSIONS

We have presented a complete survey of the state of the art in the fields of OLAP and Data Warehousing. We went from basic definitions to an in-depth study of implementation techniques, physical storage and access methods. In the process, we reviewed some topics on materialized views. This work was based on recently published papers, which are listed in the next section.

There are some items which we think will receive more attention in the

near future. Some of them are : development of efficient algorithms for data cube implementation, management of data imported from the sources(this includes consistency and duplicate checking), data compression techniques, etc.

One item we think should deserve more attention, is schema evolution. This refers to how schema changes at the sources can affect the data warehouse. Moreover, the data warehouse schema itself may change across time. This schema changes may impact many of the algorithms describe in section 4 . We believe this is a topic which has not been studied yet.

Finally, another topic on which work must be performed is modeling. Although interesting proposals are made in [AGS97] and [GL96], there is no standard query language for querying OLAP databases.

7.- REFERENCES

[AADGN96] R. Agrawal,A. Gupta,S.,S. Sarawagi, P. Deshpande, S. Agarwal, J. Naughton and R. Ramakrishnan. On the Computation of Multidimensional Aggregates. Proceedings of the 22nd VLDB Conference. Bombay, India, 1996.

[AGS97] R. Agrawal,A. Gupta and S. Sarawagi. Modeling Multidimensional Databases. IBM Almaden Research Report. 1997.

[AS] Arbor Software. Relational OLAP : Expectations and Reality. White Paper.

[CCS93] E.F. Codd, S.B. Codd and C.T. Salley. Providing OLAP (On-line Analytical Processing) to User Analysis : An IT Mandate. White Paper. Arbor Software.

[DANR96] P. Deshpande, S. Agarwal, J. Naughton and R. Ramakrishnan. Computation of Multidimensional Aggregates. Technical Report 1314. University of Wisconsin, Madison, 1996.

[GBPL97] J. Gray, A. Bosworth, A. Layman, and H. Pirahesh. Data Cube : A Relational Operator Generalizing Group-By, Cross-Tab and Sub-Totals. Data Mining and Knowledge Discovery 1, 29-53, 1997.

[GHQ95] A. Gupta, V. Harinarayan and D. Quass. Aggregate Query Processing in Data Warehousing Environments . Proceedings of the 21st VLDB Conference. Zurich, Switzerland , 1995.

[GL96] M. Gyssens and L. Lakshmanan. A Foundation for Multi-Dimensional Databases. Proceedings of the 22nd VLDB Conference. Bombay, India, 1996.

[GR96] H. Gill and P.Rao. The Official Guide to Data Warehousing. QUE Corporation, 1996.

[HRU96] V. Harinarayan, A. Rajaraman and J. Ullman. Implementing Data Cubes Efficiently. Proceedings of the 1996 ACM – SIGMOD Conference, Montreal, 1996

[IOL] Informix OnLine Extended Parallel Server and Informix Universal Server : A New Generation of Decision-Support Indexing for Enterprisewide Data Warehouses. Informix White Paper.

[K96] R. Kimball. The Data Warehouse Toolkit. J.Wiley and Sons, Inc. 1996.

[LW95] D. Lomet and J. Widom, Editors. Special Issue on Materialized Views and Data Warehousing. IEEE Data Engineering Bulletin 18(2), June 1995.

[MQM97] I. Mumick, D. Quass and B. Mumick. Maintenance of Data Cubes and Summary Tables in a Warehouse. Proceedings of the 1997 ACM – SIGMOD Conference, Tucson, Arizona, 1997.

[OCWP] OLAP Council White Paper. 1997.

[PSW] Pilot Software. An Introduction to OLAP. White Paper.

[QW97] D. Quass and J. Widom. On-line Warehouse View Maintenance. Proceedings of the 1997 ACM – SIGMOD Conference, Tucson, Arizona, 1997.

[SAG96] S. Sarawagi, R. Agrawal and A. Gupta. On Computing the Data Cube. IBM Almaden Research Report. 1996.

[SDNR96] A. Shukla, P. Deshpande, J. Naughton, K. Ramasamy. Storage Estimation for Multidimensional Aggregates in the Presence of Hierarchies. Proceedings of the 22nd VLDB Conference, Bombay, India, 1996.

[STG] Stanford Technology Group . Designing the Data Warehouse On Relational Data Warehouses. White Paper.

[W95] J. Widom. Research Problems in Data Warehousing. Proceedings of the 4th International Conference on Information and Knowledge Management, November, 1995.